

TECHNICAL DOCUMENT 0: ENVIRONMENT SETUP & PREREQUISITES GUIDE

Movie Review & Finder Web Application

This document provides a comprehensive, step-by-step onboarding guide to set up your local development environment from absolute scratch. It covers installing Git, Node.js, npm, Playwright, and setting up Docker on Windows, macOS, and Linux, including resolving common environment issues (such as unrecognized commands).

Table of Contents

1. [VS Code Onboarding: Should I Use Docker?](#)
 2. [Prerequisites Overview](#)
 3. [Step 1: Installing Git](#)
 4. [Step 2: Installing Node.js & npm \(Using Version Managers\)](#)
 5. [Step 3: Cloning the Code Repository](#)
 6. [Step 4: Installing Project Dependencies](#)
 7. [Step 5: Installing Playwright & Browser Dependencies](#)
 8. [Step 6: Docker Installation & Troubleshooting \(Fixing "Docker is not recognized"\)](#)
 9. [Step 7: Running the Application Locally](#)
-

1. VS Code Onboarding: Should I Use Docker?

Once you have cloned the project and opened the folder inside **Visual Studio Code (VS Code)**, you can run the application using either **Docker** or **Native Node.js**. Here is how to decide which to use:

Option A: Using Docker (Recommended for instant, zero-configuration setup)

- **Why use it:** You do **not** need to install Node.js, npm, or Playwright on your physical computer. Docker handles all dependency installation inside an isolated virtual container automatically!
- **Requirements:** Docker Desktop must be installed and running (see [Step 6](#)).
- **VS Code Command:**
 1. Open the integrated terminal in VS Code (**Ctrl + ``** or **Cmd + ``**).
 2. Run the single-command docker compose to build and launch the app:

```
docker compose up app --build
```

3. Open your browser and go to <http://localhost:3000>.

Option B: Native Run (Recommended for active code development)

- **Why use it:** Fastest feedback loop when editing files.

- **Requirements:** You must install Node.js, npm, and Playwright browsers on your host PC (see Steps 1-5).
- **VS Code Command:**
 1. Open the integrated terminal in VS Code (**Ctrl + ``** or **Cmd + ``**).
 2. Install all dependencies:

```
npm install
```

3. Download Playwright browser binaries:

```
npx playwright install chromium
```

4. Start the application:

```
npm run dev
```

5. Open your browser and go to <http://localhost:3000>.

2. Prerequisites Overview

To run and test the Movie Review & Finder application locally, your machine needs:

- **Git:** Version control to fetch and synchronize code.
- **Node.js:** The Javascript runtime (LTS Version 20 recommended).
- **npm:** Node Package Manager (bundled with Node.js).
- **Playwright Browsers:** Headless binaries to run E2E browser automation.
- **Docker Desktop:** For building and executing containerized workloads.

2. Step 1: Installing Git

Git is required to download (clone) the project files from your online repository.

For Windows:

1. Download the installer from the official website: git-scm.com/download/win.
2. Run the **.exe** installer. Keep the default options recommended by the wizard, ensuring **"Git from the command line and also from 3rd-party software"** is checked.
3. Open a new terminal (PowerShell or Command Prompt) and verify installation:

```
git --version
```

For macOS:

- **Option A (Using Homebrew):** If you use Homebrew, open Terminal and run:

```
brew install git
```

- **Option B (Xcode Command Line Tools):** Simply run `git` in terminal. If not installed, macOS will trigger an automatic prompt to install developer utilities.

For Linux (Ubuntu/Debian):

Run the following package commands:

```
sudo apt update
sudo apt install git -y
```

3. Step 2: Installing Node.js & npm (Using Version Managers)

We highly recommend using a **Node Version Manager** instead of direct installers. This prevents permission conflicts (``bash sudo apt update sudo apt install git -y

```
---
```

```
## 3. Step 2: Installing Node.js & npm (Using Version Managers)
```

```
We highly recommend using a Node Version Manager instead of direct installers.
This prevents permission conflicts (`EACCES` errors) and lets you switch between
project versions instantly.
```

```
### For Windows:
```

1. Download **nvm-windows** from the releases page: [github.com/coreybutler/nvm-windows](https://github.com/coreybutler/nvm-windows/releases).
2. Download and run the `nvm-setup.exe` installer.
3. Open a **new** PowerShell window as Administrator and install Node LTS (v20):
``powershell
nvm install 20
nvm use 20

4. Verify both runtime and package manager installations:

```
node -v
npm -v
```

For macOS & Linux:

1. Install **nvm** via curl command:

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh |  
bash
```

2. Restart your terminal or reload profiles:

```
source ~/.bashrc # Or ~/.zshrc for macOS
```

3. Install and set Node v20 as default:

```
nvm install 20  
nvm use 20  
nvm alias default 20
```

4. Verify versions:

```
node -v  
npm -v
```

4. Step 3: Cloning the Code Repository

Once Git is installed, navigate to the folder where you want to store the project files and run the following command to download the code:

```
# Clone the repository  
git clone <YOUR-REPOSITORY-URL>  
  
# Enter the cloned directory  
cd <REPOSITORY-FOLDER-NAME>
```

5. Step 4: Installing Project Dependencies

With your terminal pointing inside the project root folder, run npm to clean install all required packages mapped in ``bash

Clone the repository

git clone

Enter the cloned directory

cd

5. Step 4: Installing Project Dependencies

With your terminal pointing inside the project root folder, run npm to clean install all required packages mapped in `package.json`:

```
```bash
npm install
```

\*Note: If you are setting this up inside a CI/CD environment or want an exact match of the lock file, run  
``bash npm install

\*Note: If you are setting this up inside a CI/CD environment or want an exact match of the lock file, run `npm ci` instead.\*

---

## ## 6. Step 5: Installing Playwright & Browser Dependencies

Our test suite uses **Playwright** for E2E automated UI testing. Standard `npm install` does not automatically fetch the underlying browser binaries (Chromium, Firefox, WebKit). You must trigger this separately:

```
Install Browser Binaries:
```bash
npx playwright install chromium
```

(We download Chromium as it is the default targeted browser for our automation tests.)

Install Operating System Prerequisites:

If you run tests inside an isolated Linux server, a Docker container, or virtual machine and experience missing library errors (``bash npx playwright install chromium

(We download Chromium as it is the default targeted browser for our automation tests.)

Install Operating System Prerequisites:

If you run tests inside an isolated Linux server, a Docker container, or virtual machine and experience missing library errors (`libgobject`, `libgtk`, etc.), run this command to auto-install OS dependencies:

```
```bash
npx playwright install-deps
```

## 7. Step 6: Docker Installation & Troubleshooting

If you encounter this error in your terminal:

```
npx playwright install-deps
```

```

```

### ## 7. Step 6: Docker Installation & Troubleshooting

If you encounter this error in your terminal:

```
> `docker : The term 'docker' is not recognized as the name of a cmdlet...`
```

It means **Docker Desktop** is either not installed, not added to your system PATH, or the background Docker engine daemon is not running. Follow these setup instructions to resolve it.

#### ### Step 6.1: Installing Docker Desktop

##### #### A. Windows (WSL2 Architecture recommended)

1. Ensure **Hardware Virtualization** is active in your BIOS (Task Manager -> Performance tab -> CPU -> Check "Virtualization: Enabled").
2. Ensure **WSL2 (Windows Subsystem for Linux)** is active:
  - \* Open PowerShell as Administrator and run:
 

```
```powershell
wsl --install
```
```

- \* Restart your computer when prompted.

3. Download **Docker Desktop for Windows**:  
[docs.docker.com/desktop/install/windows-install](https://docs.docker.com/desktop/install/windows-install/).
4. Run the installer. Ensure the option **"Use the WSL 2 based engine (recommended)"** is checked.
5. Complete installation and restart your system.

##### #### B. macOS (Apple Silicon or Intel)

1. Download **Docker Desktop for Mac**: [docs.docker.com/desktop/install/mac-install](https://docs.docker.com/desktop/install/mac-install/). Choose the correct version (Apple Silicon/M1/M2/M3 vs Intel Chip).
2. Open the downloaded `.dmg` file and drag Docker to your /Applications` directory.`

```

```

### ### Step 6.2: Launch and Verify the Docker Engine

1. Open the **Docker Desktop** application from your OS search bar.
2. Accept the service agreements.
3. Wait for the indicator in the bottom-left corner of the Docker window to turn **Green (Running)**.
4. **CRITICAL**: Close all active terminals/IDE instances and open a **fresh** PowerShell, Command Prompt, or Zsh terminal. (PATH updates require a new shell instance to be registered).
5. Verify the CLI exposure:

```
```bash
docker --version
docker compose version
```

6. Run a diagnostic container to confirm everything works:

```
docker run hello-world
```

8. Step 7: Running the Application Locally

Now that everything is installed, choose your preferred method to execute the web application and run the verification tests.

Method A: Single-Command Docker Compose (Easiest & Cleanest)

Using **Docker Compose**, you do not need to manually run separate build and execution commands. Docker Compose compiles the images and starts the container in a single line!

- **To run the Full Production App:**

```
docker compose up app --build
```

Access in your browser at: <http://localhost:3000>

- **To run the Complete CI/CD Test Automation Pipeline:**

```
docker compose up pipeline --build
```

This builds the test image, triggers dependency install, code-building, starts the webserver, waits for it to become healthy, and executes all Playwright E2E browser tests automatically inside a clean, isolated Linux container!

9. Alternative Run Methods (Manual Setup)

If you prefer to run development processes outside of Docker Compose, you can utilize standard npm or manual Docker CLI commands:

Method B: Native Dev Execution (Recommended for Fast Changes)

```
# 1. Start backend server and Vite frontend concurrently
npm run dev

# 2. View in browser: http://localhost:3000
```

Method C: Native E2E Test Execution

```
# Ensure dev server is not running on port 3000, then run:
npm run test:e2e
```

Method D: Running via Manual Docker CLI Commands

If you prefer not to use Docker Compose, you can build and run individual containers manually:

- **For the Production App:**

```
docker build -t movie-app-prod -f Dockerfile .
docker run -p 3000:3000 movie-app-prod
```

- **For the CI/CD Pipeline Container:**

```
docker build -t movie-app-cicd -f Dockerfile.cicd .
docker run -it -p 3000:3000 movie-app-cicd
```